

class-connect- - PDF 2 - tests/config

Source-code export | 2026-09-10 | landscape A4 | monospace | selectable text

Contents:

```
=== backend/test/helpers.js ===  
=== backend/test/api.test.js ===  
=== backend/test/validation.test.js ===  
    === package.json ===  
    === .github/workflows/ci.yml ===  
    === .github/workflows/deploy.yml ===
```

Line numbers shown at left; long lines wrap with a '+' continuation marker. No secrets are included.

=== backend/test/helpers.js === (271 lines, 8167 bytes)

```
1 | import { DatabaseSync } from "node:sqlite";
2 | import { readFileSync } from "node:fs";
3 | import worker, { Classroom } from "../src/index.js";
4 |
5 | export const ORIGIN = "http://localhost";
6 | export const COOKIE_NAME = "classconnect-dev";
7 |
8 | const MIGRATION = readFileSync(
9 |   new URL("../migrations/0001_initial.sql", import.meta.url),
10 |   "utf8"
11 | );
12 |
13 | /** Minimal `ctx.storage.sql` shim over node:sqlite for Durable Object classes. */
14 | function makeSql(db) {
15 |   return {
16 |     exec(query, ...params) {
17 |       const head = query.trim().slice(0, 6).toUpperCase();
18 |       if (params.length === 0 && head !== "SELECT") {
19 |         db.exec(query); // multi-statement DDL/DML without parameters
20 |         return [];
21 |       }
22 |       const stmt = db.prepare(query);
23 |       if (head === "SELECT") return stmt.all(...params);
24 |       stmt.run(...params);
25 |       return [];
26 |     }
27 |   };
28 | }
29 |
30 | const asNumber = value => (typeof value === "bigint" ? Number(value) : value);
31 |
32 | /** D1-compatible shim (`prepare`/`bind`/`first`/`all`/`run`/`batch`) over node:sqlite. */
33 | function makeD1(db) {
34 |   return {
35 |     prepare(query) {
36 |       const stmt = db.prepare(query);
37 |       return {
38 |         bind(...params) {
39 |           return {
40 |             first(column) {
41 |               const row = stmt.get(...params) ?? null;
42 |               if (row === null) return null;
43 |               return column === undefined ? row : (row[column] ?? null);
44 |             },
45 |             all() {
46 |               return {
47 |                 success: true,
48 |                 results: stmt.all(...params),
49 |                 meta: { rows_read: 0, rows_written: 0 }
50 |               };
51 |             },
52 |             run() {
53 |               const info = stmt.run(...params);
54 |               return {
55 |                 success: true,
56 |                 meta: {
57 |                   changes: asNumber(info.changes),
```

```

58 |         last_row_id: asNumber(info.lastInsertRowid),
59 |         rows_read: 0,
60 |         rows_written: 0
61 |     }
62 | };
63 | }
64 | };
65 | }
66 | };
67 | },
68 | async batch(statements) {
69 |     const results = [];
70 |     for (const statement of statements) {
71 |         results.push(await statement.run());
72 |     }
73 |     return results;
74 | }
75 | };
76 | }
77 |
78 | /** In-memory R2 shim. */
79 | function makeR2() {
80 |     const objects = new Map();
81 |     return {
82 |         async put(key, value, options = {}) {
83 |             let bytes;
84 |             if (typeof value === "string") bytes = new TextEncoder().encode(value);
85 |             else if (value instanceof Uint8Array) bytes = value.slice();
86 |             else if (value instanceof ArrayBuffer) bytes = new Uint8Array(value.slice(0));
87 |             else throw new Error("Unsupported R2 value type in tests");
88 |             objects.set(key, { bytes, httpMetadata: options.httpMetadata ?? {} });
89 |         },
90 |         async get(key) {
91 |             const object = objects.get(key);
92 |             if (!object) return null;
93 |             return { body: object.bytes.slice(), size: object.bytes.byteLength };
94 |         },
95 |         async delete(key) {
96 |             objects.delete(key);
97 |         },
98 |         /** Test-only introspection. */
99 |         __keys: () => [...objects.keys()]
100 |     };
101 | }
102 |
103 | export async function sha256hex(value) {
104 |     const digest = await crypto.subtle.digest(
105 |         "SHA-256",
106 |         new TextEncoder().encode(value)
107 |     );
108 |     return [...new Uint8Array(digest)]
109 |         .map(byte => byte.toString(16).padStart(2, "0"))
110 |         .join("");
111 | }
112 |
113 | /**
114 |  * Build a fresh worker `env` backed by an empty in-memory database.
115 |  * Quotas are allowed by default; flip `env.__test.guard.allowed` to
116 |  * exercise 429 paths. Room routes run through the real Classroom class.

```

```

117 |  */
118 | export function createTestEnv() {
119 |   const db = new DatabaseSync(":memory:");
120 |   db.exec("PRAGMA foreign_keys = ON;");
121 |   db.exec(MIGRATION);
122 |
123 |   const guard = { allowed: true };
124 |   const roomDb = new DatabaseSync(":memory:");
125 |   let env;
126 |
127 |   const roomCtx = {
128 |     storage: { sql: makeSql(roomDb) },
129 |     getWebSockets: () => [],
130 |     acceptWebSocket: () => {}
131 |   };
132 |
133 |   env = {
134 |     APP_ORIGIN: ORIGIN,
135 |     DB: makeD1(db),
136 |     FILES: makeR2(),
137 |     GUARD: {
138 |       idFromName: name => name,
139 |       get: () => ({
140 |         fetch: async () =>
141 |           guard.allowed
142 |             ? new Response("OK")
143 |             : new Response("Quota exceeded", { status: 429 })
144 |       })
145 |     },
146 |     ROOMS: null, // wired below, after `env` exists
147 |     __test: { db, guard }
148 |   };
149 |
150 |   const rooms = new Map();
151 |   const room = id => {
152 |     if (!rooms.has(id)) rooms.set(id, new Classroom(roomCtx, env));
153 |     return rooms.get(id);
154 |   };
155 |   env.ROOMS = {
156 |     idFromName: name => name,
157 |     get: id => ({ fetch: request => room(String(id)).fetch(request) })
158 |   };
159 |
160 |   return env;
161 | }
162 |
163 | /** Build a Request against the test worker (sets Origin for unsafe methods). */
164 | export function apiRequest(
165 |   path,
166 |   { method = "GET", body, cookie, origin = ORIGIN, headers = {} } = {}
167 | ) {
168 |   const requestHeaders = { ...headers };
169 |   if (!["GET", "HEAD", "OPTIONS"].includes(method)) {
170 |     requestHeaders.Origin = origin;
171 |   }
172 |   if (cookie) requestHeaders.Cookie = cookie;
173 |
174 |   let payload;
175 |   if (body !== undefined) {

```

```

176 |     if (typeof body === "string" || body instanceof Uint8Array) {
177 |       payload = body;
178 |     } else {
179 |       payload = JSON.stringify(body);
180 |       requestHeaders["Content-Type"] = "application/json";
181 |     }
182 |   }
183 |
184 |   return new Request(ORIGIN + path, {
185 |     method,
186 |     headers: requestHeaders,
187 |     body: payload
188 |   });
189 | }
190 |
191 | export async function callWorker(env, path, options) {
192 |   const response = await worker.fetch(apiRequest(path, options), env);
193 |   let parsed = null;
194 |   try {
195 |     parsed = await response.clone().json();
196 |   } catch {
197 |     parsed = null;
198 |   }
199 |   return { status: response.status, headers: response.headers, json: parsed, response };
200 | }
201 |
202 | /** Extract the `name=token` Cookie header value from a Set-Cookie response. */
203 | export function sessionCookie(response) {
204 |   const setCookie =
205 |     typeof response.headers.getSetCookie === "function"
206 |       ? response.headers.getSetCookie()[0]
207 |       : response.headers.get("Set-Cookie");
208 |   if (!setCookie) return null;
209 |   return setCookie.split(";")[0];
210 | }
211 |
212 | /** Seed a user row directly; returns the row plus its recovery key. */
213 | export async function seedUser(env, { name = "Test User", role = "teacher" } = {}) {
214 |   const id = crypto.randomUUID();
215 |   const recoveryKey = [...crypto.getRandomValues(new Uint8Array(32))]
216 |     .map(byte => byte.toString(16).padStart(2, "0"))
217 |     .join("");
218 |   env.__test.db
219 |     .prepare(
220 |       "INSERT INTO users(id, display_name, role, recovery_hash, created_at) VALUES (?, ?, ?, ?, ?)"
221 |     )
222 |     .run(id, name, role, await sha256hex(recoveryKey), Date.now());
223 |   return { id, name, role, recoveryKey };
224 | }
225 |
226 | /** Log a seeded user in through the real /api/auth/login route. */
227 | export async function login(env, recoveryKey) {
228 |   const result = await callWorker(env, "/api/auth/login", {
229 |     method: "POST",
230 |     body: { key: recoveryKey }
231 |   });
232 |   if (result.status !== 200) {
233 |     throw new Error(`login failed: ${result.status} ${JSON.stringify(result.json)}`);
234 |   }

```

```

235 |   return { user: result.json.user, cookie: sessionCookie(result.response) };
236 | }
237 |
238 | /** Create a class through the real route; returns { id, name, code }. */
239 | export async function createClass(env, cookie, name = "Physics 101") {
240 |   const result = await callWorker(env, "/api/classes", {
241 |     method: "POST",
242 |     cookie,
243 |     body: { name }
244 |   });
245 |   if (result.status !== 201) {
246 |     throw new Error(
247 |       `createClass failed: ${result.status} ${JSON.stringify(result.json)}`
248 |     );
249 |   }
250 |   return result.json;
251 | }
252 |
253 | /** Full baseline: teacher + login + class. */
254 | export async function setupClassroom(env) {
255 |   const teacher = await seedUser(env, { name: "Ms. Rahman", role: "teacher" });
256 |   const session = await login(env, teacher.recoveryKey);
257 |   const classroom = await createClass(env, session.cookie);
258 |   return { teacher, cookie: session.cookie, classroom };
259 | }
260 |
261 | /** Enroll a student through the real /api/auth/join route. */
262 | export async function joinClass(env, code, name = "Arif") {
263 |   const result = await callWorker(env, "/api/auth/join", {
264 |     method: "POST",
265 |     body: { name, code }
266 |   });
267 |   if (result.status !== 201) {
268 |     throw new Error(`joinClass failed: ${result.status} ${JSON.stringify(result.json)}`);
269 |   }
270 |   return { user: result.json.user, cookie: sessionCookie(result.response) };
271 | }

```

=== backend/test/api.test.js === (590 lines, 19234 bytes)

```
1 | import test from "node:test";
2 | import assert from "node:assert/strict";
3 |
4 | import worker from "../src/index.js";
5 | import {
6 |   ORIGIN,
7 |   COOKIE_NAME,
8 |   callWorker,
9 |   createClass,
10 |   createTestEnv,
11 |   joinClass,
12 |   login,
13 |   seedUser,
14 |   sessionCookie,
15 |   setupClassroom,
16 |   sha256hex
17 | } from "../helpers.js";
18 |
19 | // WebSocket upgrade paths (/ws/...) need the Workers runtime's WebSocketPair
20 | // and are intentionally not covered here; every HTTP route is.
21 |
22 | const pngBytes = () =>
23 |   new Uint8Array([137, 80, 78, 71, 13, 10, 26, 10, 0, 0, 0, 1, 2, 3, 4]);
24 |
25 | test("GET /api/health reports status and fixed feature flags", async () => {
26 |   const env = createTestEnv();
27 |   const result = await callWorker(env, "/api/health");
28 |
29 |   assert.equal(result.status, 200);
30 |   assert.deepEqual(result.json, {
31 |     ok: true,
32 |     features: { push: false, calls: false, directMessages: false }
33 |   });
34 | });
35 |
36 | test("requests from an unrecognized origin are rejected", async () => {
37 |   const env = createTestEnv();
38 |
39 |   const wrongHost = await worker.fetch(
40 |     new Request("http://evil.example/api/health"),
41 |     env
42 |   );
43 |   assert.equal(wrongHost.status, 403);
44 |
45 |   const missingOrigin = await worker.fetch(
46 |     new Request(`${ORIGIN}/api/auth/login`, {
47 |       method: "POST",
48 |       headers: { "Content-Type": "application/json" },
49 |       body: JSON.stringify({ key: "x".repeat(64) })
50 |     ),
51 |     env
52 |   );
53 |   assert.equal(missingOrigin.status, 403);
54 | });
55 |
56 | test("login validates the recovery key before touching the database", async () => {
57 |   const env = createTestEnv();
```

```

58 |
59 |   const missing = await callWorker(env, "/api/auth/login", {
60 |     method: "POST",
61 |     body: {}
62 |   });
63 |   assert.equal(missing.status, 400);
64 |
65 |   const malformed = await callWorker(env, "/api/auth/login", {
66 |     method: "POST",
67 |     body: { key: "not-a-key" }
68 |   });
69 |   assert.equal(malformed.status, 400);
70 |
71 |   const unknown = await callWorker(env, "/api/auth/login", {
72 |     method: "POST",
73 |     body: { key: "a".repeat(64) }
74 |   });
75 |   assert.equal(unknown.status, 401);
76 | });
77 |
78 | test("teacher login issues a session cookie usable for /api/me", async () => {
79 |   const env = createTestEnv();
80 |   const teacher = await seedUser(env, { name: "Ms. Rahman", role: "teacher" });
81 |
82 |   const result = await callWorker(env, "/api/auth/login", {
83 |     method: "POST",
84 |     body: { key: teacher.recoveryKey }
85 |   });
86 |
87 |   assert.equal(result.status, 200);
88 |   assert.deepEqual(result.json.user, {
89 |     id: teacher.id,
90 |     display_name: "Ms. Rahman",
91 |     role: "teacher"
92 |   });
93 |
94 |   const cookie = sessionCookie(result.response);
95 |   assert.ok(cookie.startsWith(`${COOKIE_NAME}=`));
96 |
97 |   const me = await callWorker(env, "/api/me", { cookie });
98 |   assert.equal(me.status, 200);
99 |   assert.equal(me.json.user.id, teacher.id);
100 | });
101 |
102 | test("authenticated routes reject missing, malformed and unknown sessions", async () => {
103 |   const env = createTestEnv();
104 |
105 |   const missing = await callWorker(env, "/api/me");
106 |   assert.equal(missing.status, 401);
107 |
108 |   const malformed = await callWorker(env, "/api/me", {
109 |     cookie: `${COOKIE_NAME}=not-a-token`
110 |   });
111 |   assert.equal(malformed.status, 401);
112 |
113 |   const unknown = await callWorker(env, "/api/me", {
114 |     cookie: `${COOKIE_NAME}=${"b".repeat(64)}`
115 |   });
116 |   assert.equal(unknown.status, 401);

```

```

117 | });
118 |
119 | test("logout destroys the session server-side", async () => {
120 |   const env = createTestEnv();
121 |   const { cookie } = await setupClassroom(env);
122 |
123 |   const logout = await callWorker(env, "/api/auth/logout", {
124 |     method: "POST",
125 |     cookie
126 |   });
127 |   assert.equal(logout.status, 200);
128 |
129 |   const me = await callWorker(env, "/api/me", { cookie });
130 |   assert.equal(me.status, 401);
131 | });
132 |
133 | test("teachers can create classes and list them; students cannot create", async () => {
134 |   const env = createTestEnv();
135 |   const { cookie, classroom } = await setupClassroom(env);
136 |
137 |   assert.equal(classroom.name, "Physics 101");
138 |   assert.match(classroom.code, /^[A-F0-9]{20}$/);
139 |
140 |   const list = await callWorker(env, "/api/classes", { cookie });
141 |   assert.equal(list.status, 200);
142 |   assert.equal(list.json.classes.length, 1);
143 |   assert.equal(list.json.classes[0].id, classroom.id);
144 |
145 |   const student = await joinClass(env, classroom.code, "Arif");
146 |   const denied = await callWorker(env, "/api/classes", {
147 |     method: "POST",
148 |     cookie: student.cookie,
149 |     body: { name: "Sneaky class" }
150 |   });
151 |   assert.equal(denied.status, 403);
152 | });
153 |
154 | test("students enroll with an invitation code; bad codes are rejected", async () => {
155 |   const env = createTestEnv();
156 |   const { classroom } = await setupClassroom(env);
157 |
158 |   const bad = await callWorker(env, "/api/auth/join", {
159 |     method: "POST",
160 |     body: { name: "Nope", code: "INVALID-CODE-1" }
161 |   });
162 |   assert.equal(bad.status, 400);
163 |
164 |   // Codes are normalized (case/dashes/whitespace insensitive).
165 |   const code = classroom.code.toLowerCase();
166 |   const dashed = `${code.slice(0, 4)}-${code.slice(4, 8)} ${code.slice(8)}`;
167 |   const joined = await callWorker(env, "/api/auth/join", {
168 |     method: "POST",
169 |     body: { name: " Arif ", code: dashed }
170 |   });
171 |
172 |   assert.equal(joined.status, 201);
173 |   assert.equal(joined.json.user.display_name, "Arif");
174 |   assert.equal(joined.json.user.role, "student");
175 |   assert.match(joined.json.recoveryKey, /^[a-f0-9]{64}$/);

```

```

176 |   assert.ok(sessionCookie(joined.response));
177 |
178 |   // The recovery key works for later logins.
179 |   const again = await login(env, joined.json.recoveryKey);
180 |   assert.equal(again.user.display_name, "Arif");
181 | });
182 |
183 | test("existing users can join more classes with a code", async () => {
184 |   const env = createTestEnv();
185 |   const { cookie, classroom } = await setupClassroom(env);
186 |   const second = await createClass(env, cookie, "Chemistry 101");
187 |
188 |   const student = await joinClass(env, classroom.code, "Arif");
189 |   const joined = await callWorker(env, "/api/classes/join", {
190 |     method: "POST",
191 |     cookie: student.cookie,
192 |     body: { code: second.code }
193 |   });
194 |
195 |   assert.equal(joined.status, 200);
196 |   assert.equal(joined.json.id, second.id);
197 |
198 |   const list = await callWorker(env, "/api/classes", { cookie: student.cookie });
199 |   assert.equal(list.json.classes.length, 2);
200 |
201 |   const bad = await callWorker(env, "/api/classes/join", {
202 |     method: "POST",
203 |     cookie: student.cookie,
204 |     body: { code: "INVALID-CODE-1" }
205 |   });
206 |   assert.equal(bad.status, 400);
207 | });
208 |
209 | test("uploads validate membership, size and file contents", async () => {
210 |   const env = createTestEnv();
211 |   const { cookie, classroom } = await setupClassroom(env);
212 |   const outsider = await seedUser(env, { name: "Outsider", role: "student" });
213 |   const outsiderSession = await login(env, outsider.recoveryKey);
214 |
215 |   const upload = (bytes, cookieValue, size = bytes.byteLength, mime = "image/png") =>
216 |     callWorker(
217 |       env,
218 |       `/api/uploads?class=${classroom.id}&name=photo.png&size=${size}`,
219 |       { method: "POST", cookie: cookieValue, body: bytes, headers: { "Content-Type": mime } }
220 |     );
221 |
222 |   const ok = await upload(pngBytes(), cookie);
223 |   assert.equal(ok.status, 201);
224 |   assert.equal(ok.json.file_name, "photo.png");
225 |   assert.equal(ok.json.mime_type, "image/png");
226 |
227 |   const nonMember = await upload(pngBytes(), outsiderSession.cookie);
228 |   assert.equal(nonMember.status, 403);
229 |
230 |   const sizeMismatch = await upload(pngBytes(), cookie, 999);
231 |   assert.equal(sizeMismatch.status, 400);
232 |
233 |   const badMime = await upload(pngBytes(), cookie, pngBytes().byteLength, "text/html");
234 |   assert.equal(badMime.status, 400);

```

```

235 |
236 |   const forged = await upload(new Uint8Array([1, 2, 3, 4, 5, 6, 7, 8]), cookie);
237 |   assert.equal(forged.status, 400);
238 | });
239 |
240 | test("attachments download only for members once attached to a message", async () => {
241 |   const env = createTestEnv();
242 |   const { teacher, cookie, classroom } = await setupClassroom(env);
243 |
244 |   const uploaded = await callWorker(
245 |     env,
246 |     `/api/uploads?class=${classroom.id}&name=photo.png&size=${pngBytes().byteLength}`,
247 |     {
248 |       method: "POST",
249 |       cookie,
250 |       body: pngBytes(),
251 |       headers: { "Content-Type": "image/png" }
252 |     }
253 |   );
254 |   const attachmentId = uploaded.json.id;
255 |
256 |   // Orphaned uploads are not downloadable.
257 |   const orphan = await callWorker(env, `/api/files/${attachmentId}`, { cookie });
258 |   assert.equal(orphan.status, 404);
259 |
260 |   const messageId = crypto.randomUUID();
261 |   env.__test.db
262 |     .prepare(
263 |       "INSERT INTO messages(id, class_id, sender_id, client_id, content, attachment_id, created_at) VALUES (?, ?, ?, ?, ?, ?, ?)"
264 |     )
265 |     .run(messageId, classroom.id, teacher.id, "c1", "", attachmentId, Date.now());
266 |
267 |   const file = await callWorker(env, `/api/files/${attachmentId}`, { cookie });
268 |   assert.equal(file.status, 200);
269 |   assert.equal(file.response.headers.get("Content-Type"), "image/png");
270 |   assert.match(
271 |     file.response.headers.get("Content-Disposition"),
272 |     /^attachment; filename\*=UTF-8'photo\.png$/
273 |   );
274 |   assert.deepEqual(new Uint8Array(await file.response.arrayBuffer()), pngBytes());
275 |
276 |   const outsider = await seedUser(env, { name: "Outsider", role: "student" });
277 |   const outsiderSession = await login(env, outsider.recoveryKey);
278 |   const denied = await callWorker(env, `/api/files/${attachmentId}`, {
279 |     cookie: outsiderSession.cookie
280 |   });
281 |   assert.equal(denied.status, 404);
282 |
283 |   const missing = await callWorker(env, `/api/files/${crypto.randomUUID()}`, {
284 |     cookie
285 |   });
286 |   assert.equal(missing.status, 404);
287 | });
288 |
289 | test("roster and message history are visible to members only", async () => {
290 |   const env = createTestEnv();
291 |   const { teacher, cookie, classroom } = await setupClassroom(env);
292 |   const student = await joinClass(env, classroom.code, "Arif");
293 |

```

```

294 |   const roster = await callWorker(env, `/api/classes/${classroom.id}/roster`, {
295 |     cookie
296 |   });
297 |   assert.equal(roster.status, 200);
298 |   assert.deepEqual(
299 |     roster.json.members.map(member => member.display_name).sort(),
300 |     ["Arif", "Ms. Rahman"]
301 |   );
302 |
303 |   const db = env.__test.db;
304 |   const insert = db.prepare(
305 |     "INSERT INTO messages(id, class_id, sender_id, client_id, content, created_at) VALUES (?, ?, ?, ?, ?, ?)"
306 |   );
307 |   insert.run(crypto.randomUUID(), classroom.id, teacher.id, "t1", "Welcome", Date.now());
308 |   insert.run(crypto.randomUUID(), classroom.id, student.user.id, "s1", "Hello", Date.now());
309 |
310 |   const history = await callWorker(env, `/api/classes/${classroom.id}/messages`, {
311 |     cookie: student.cookie
312 |   });
313 |   assert.equal(history.status, 200);
314 |   assert.deepEqual(
315 |     history.json.messages.map(message => message.content),
316 |     ["Welcome", "Hello"]
317 |   );
318 |   assert.ok(history.json.messages[0].seq < history.json.messages[1].seq);
319 |
320 |   const after = await callWorker(
321 |     env,
322 |     `/api/classes/${classroom.id}/messages?after=${history.json.messages[0].seq}`,
323 |     { cookie }
324 |   );
325 |   assert.equal(after.json.messages.length, 1);
326 |   assert.equal(after.json.messages[0].content, "Hello");
327 |
328 |   const invalidCursor = await callWorker(
329 |     env,
330 |     `/api/classes/${classroom.id}/messages?after=nope`,
331 |     { cookie }
332 |   );
333 |   assert.equal(invalidCursor.status, 400);
334 |
335 |   const outsider = await seedUser(env, { name: "Outsider", role: "student" });
336 |   const outsiderSession = await login(env, outsider.recoveryKey);
337 |   const denied = await callWorker(env, `/api/classes/${classroom.id}/roster`, {
338 |     cookie: outsiderSession.cookie
339 |   });
340 |   assert.equal(denied.status, 403);
341 | });
342 |
343 | test("only the class teacher can rotate invites, mute, remove and delete", async () => {
344 |   const env = createTestEnv();
345 |   const { cookie, classroom } = await setupClassroom(env);
346 |   const student = await joinClass(env, classroom.code, "Arif");
347 |
348 |   const invite = await callWorker(env, `/api/classes/${classroom.id}/invite`, {
349 |     method: "POST",
350 |     cookie: student.cookie,
351 |     body: {}
352 |   });

```

```

353 |   assert.equal(invite.status, 403);
354 |
355 |   const mute = await callWorker(
356 |     env,
357 |     `/api/classes/${classroom.id}/members/${student.user.id}`,
358 |     { method: "PATCH", cookie: student.cookie, body: { muted: true } }
359 |   );
360 |   assert.equal(mute.status, 403);
361 |
362 |   const remove = await callWorker(
363 |     env,
364 |     `/api/classes/${classroom.id}/members/${student.user.id}`,
365 |     { method: "DELETE", cookie: student.cookie }
366 |   );
367 |   assert.equal(remove.status, 403);
368 |
369 |   const erase = await callWorker(
370 |     env,
371 |     `/api/classes/${classroom.id}/messages/${crypto.randomUUID()}`,
372 |     { method: "DELETE", cookie: student.cookie }
373 |   );
374 |   assert.equal(erase.status, 403);
375 | });
376 |
377 | test("teachers can mute and unmute students; muted students cannot upload", async () => {
378 |   const env = createTestEnv();
379 |   const { cookie, classroom } = await setupClassroom(env);
380 |   const student = await joinClass(env, classroom.code, "Arif");
381 |
382 |   const mute = await callWorker(
383 |     env,
384 |     `/api/classes/${classroom.id}/members/${student.user.id}`,
385 |     { method: "PATCH", cookie, body: { muted: true } }
386 |   );
387 |   assert.equal(mute.status, 200);
388 |
389 |   const roster = await callWorker(env, `/api/classes/${classroom.id}/roster`, {
390 |     cookie
391 |   });
392 |   const entry = roster.json.members.find(member => member.id === student.user.id);
393 |   assert.equal(entry.muted, 1);
394 |
395 |   const blocked = await callWorker(
396 |     env,
397 |     `/api/uploads?class=${classroom.id}&name=photo.png&size=${pngBytes().byteLength}`,
398 |     {
399 |       method: "POST",
400 |       cookie: student.cookie,
401 |       body: pngBytes(),
402 |       headers: { "Content-Type": "image/png" }
403 |     }
404 |   );
405 |   assert.equal(blocked.status, 403);
406 |
407 |   const unmute = await callWorker(
408 |     env,
409 |     `/api/classes/${classroom.id}/members/${student.user.id}`,
410 |     { method: "PATCH", cookie, body: { muted: false } }
411 |   );

```

```

412 |   assert.equal(unmute.status, 200);
413 |
414 |   const allowed = await callWorker(
415 |     env,
416 |     `/api/uploads?class=${classroom.id}&name=photo.png&size=${pngBytes().byteLength}`,
417 |     {
418 |       method: "POST",
419 |       cookie: student.cookie,
420 |       body: pngBytes(),
421 |       headers: { "Content-Type": "image/png" }
422 |     }
423 |   );
424 |   assert.equal(allowed.status, 201);
425 | });
426 |
427 | test("rotating the invitation invalidates the old code", async () => {
428 |   const env = createTestEnv();
429 |   const { cookie, classroom } = await setupClassroom(env);
430 |
431 |   const rotated = await callWorker(env, `/api/classes/${classroom.id}/invite`, {
432 |     method: "POST",
433 |     cookie,
434 |     body: {}
435 |   });
436 |   assert.equal(rotated.status, 200);
437 |   assert.match(rotated.json.code, /^[A-F0-9]{20}$/);
438 |   assert.notEqual(rotated.json.code, classroom.code);
439 |
440 |   const stale = await callWorker(env, "/api/auth/join", {
441 |     method: "POST",
442 |     body: { name: "Late", code: classroom.code }
443 |   });
444 |   assert.equal(stale.status, 400);
445 |
446 |   const fresh = await joinClass(env, rotated.json.code, "OnTime");
447 |   assert.equal(fresh.user.display_name, "OnTime");
448 | });
449 |
450 | test("teachers can delete messages; history shows a tombstone", async () => {
451 |   const env = createTestEnv();
452 |   const { teacher, cookie, classroom } = await setupClassroom(env);
453 |   const messageId = crypto.randomUUID();
454 |
455 |   env.__test.db
456 |     .prepare(
457 |       "INSERT INTO messages(id, class_id, sender_id, client_id, content, created_at) VALUES (?, ?, ?, ?, ?, ?)"
458 |     )
459 |     .run(messageId, classroom.id, teacher.id, "t1", "Oops", Date.now());
460 |
461 |   const erased = await callWorker(
462 |     env,
463 |     `/api/classes/${classroom.id}/messages/${messageId}`,
464 |     { method: "DELETE", cookie }
465 |   );
466 |   assert.equal(erased.status, 200);
467 |
468 |   const history = await callWorker(env, `/api/classes/${classroom.id}/messages`, {
469 |     cookie
470 |   });

```

```

471 |   assert.equal(history.json.messages.length, 1);
472 |   assert.equal(history.json.messages[0].content, "");
473 |   assert.equal(history.json.messages[0].deleted, true);
474 | });
475 |
476 | test("removing a member revokes access, rotates the code and blocks rejoin", async () => {
477 |   const env = createTestEnv();
478 |   const { cookie, classroom } = await setupClassroom(env);
479 |   const student = await joinClass(env, classroom.code, "Arif");
480 |
481 |   const removed = await callWorker(
482 |     env,
483 |     `/api/classes/${classroom.id}/members/${student.user.id}`,
484 |     { method: "DELETE", cookie }
485 |   );
486 |   assert.equal(removed.status, 200);
487 |   assert.match(removed.json.code, /^[A-F0-9]{20}$/);
488 |
489 |   const roster = await callWorker(env, `/api/classes/${classroom.id}/roster`, {
490 |     cookie: student.cookie
491 |   });
492 |   assert.equal(roster.status, 403);
493 |
494 |   const rejoin = await callWorker(env, "/api/classes/join", {
495 |     method: "POST",
496 |     cookie: student.cookie,
497 |     body: { code: removed.json.code }
498 |   });
499 |   assert.equal(rejoin.status, 403);
500 |
501 |   const stale = await callWorker(env, "/api/auth/join", {
502 |     method: "POST",
503 |     body: { name: "Late", code: classroom.code }
504 |   });
505 |   assert.equal(stale.status, 400);
506 |
507 |   // Brand-new students can still enroll with the rotated code.
508 |   const fresh = await joinClass(env, removed.json.code, "Newbie");
509 |   assert.equal(fresh.user.display_name, "Newbie");
510 | });
511 |
512 | test("quota exhaustion surfaces as HTTP 429", async () => {
513 |   const env = createTestEnv();
514 |   const { cookie } = await setupClassroom(env);
515 |
516 |   env.__test.guard.allowed = false;
517 |   try {
518 |     const limited = await callWorker(env, "/api/classes", {
519 |       method: "POST",
520 |       cookie,
521 |       body: { name: "Over quota" }
522 |     });
523 |     assert.equal(limited.status, 429);
524 |   } finally {
525 |     env.__test.guard.allowed = true;
526 |   }
527 | });
528 |
529 | test("unknown routes return 404", async () => {

```

```

530 |   const env = createTestEnv();
531 |   const { cookie } = await setupClassroom(env);
532 |
533 |   const missing = await callWorker(env, "/api/nope", { cookie });
534 |   assert.equal(missing.status, 404);
535 |   assert.equal(missing.json.error, "Not found.");
536 | });
537 |
538 | test("scheduled cleanup purges expired sessions, old messages and stale uploads", async () => {
539 |   const env = createTestEnv();
540 |   const { teacher, classroom } = await setupClassroom(env);
541 |   const db = env.__test.db;
542 |   const day = 86400000;
543 |   const now = Date.now();
544 |
545 |   const staleKey = "stale-object";
546 |   await env.FILES.put(staleKey, new Uint8Array([1, 2, 3]));
547 |   db.prepare(
548 |     "INSERT INTO attachments(id, class_id, owner_id, r2_key, file_name, mime_type, size_bytes, created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?)"
549 |   ).run(crypto.randomUUID(), classroom.id, teacher.id, staleKey, "old.png", "image/png", 3, now - 2 * day);
550 |
551 |   db.prepare(
552 |     "INSERT INTO messages(id, class_id, sender_id, client_id, content, created_at) VALUES (?, ?, ?, ?, ?, ?)"
553 |   ).run(crypto.randomUUID(), classroom.id, teacher.id, "old", "ancient", now - 31 * day);
554 |
555 |   db.prepare(
556 |     "INSERT INTO sessions(token_hash, user_id, expires_at) VALUES (?, ?, ?)"
557 |   ).run(await sha256hex("expired"), teacher.id, now - 1000);
558 |
559 |   const pending = [];
560 |   await worker.scheduled({}, env, { waitUntil: promise => pending.push(promise) });
561 |   await Promise.all(pending);
562 |
563 |   assert.equal(env.FILES.__keys().length, 0);
564 |   assert.equal(
565 |     db.prepare("SELECT COUNT(*) AS n FROM attachments").get().n,
566 |     0
567 |   );
568 |   assert.equal(db.prepare("SELECT COUNT(*) AS n FROM messages").get().n, 0);
569 |   assert.equal(
570 |     db
571 |       .prepare("SELECT COUNT(*) AS n FROM sessions WHERE token_hash = ?")
572 |       .get(await sha256hex("expired")).n,
573 |     0
574 |   );
575 |   // The teacher's live session survives.
576 |   assert.ok(db.prepare("SELECT COUNT(*) AS n FROM sessions").get().n >= 1);
577 | });
578 |
579 | test("failed requests still set JSON content type and security headers", async () => {
580 |   const env = createTestEnv();
581 |   const result = await callWorker(env, "/api/me");
582 |
583 |   assert.equal(result.status, 401);
584 |   assert.match(
585 |     result.response.headers.get("Content-Type"),
586 |     /^application\/json; charset=utf-8$/
587 |   );
588 |   assert.equal(result.response.headers.get("X-Content-Type-Options"), "nosniff");

```

```
589 |     assert.equal(result.response.headers.get("Cache-Control"), "no-store");
590 | });
```

=== backend/test/validation.test.js === (34 lines, 941 bytes)

```
1 | import test from "node:test";
2 | import assert from "node:assert/strict";
3 |
4 | import {
5 |   cleanText,
6 |   normalizeInvite,
7 |   allowedMime
8 | } from "../src/index.js";
9 |
10 | test("trims valid display names", () => {
11 |   assert.equal(cleanText(" Priya ", 1, 60, "Name"), "Priya");
12 | });
13 |
14 | test("rejects empty names", () => {
15 |   assert.throws(() => cleanText(" ", 1, 60, "Name"));
16 | });
17 |
18 | test("rejects oversized messages", () => {
19 |   assert.throws(() => cleanText("x".repeat(4001), 0, 4000, "Message"));
20 | });
21 |
22 | test("normalizes invitation codes", () => {
23 |   assert.equal(normalizeInvite("abcd-efgh"), "ABCDEFGH");
24 | });
25 |
26 | test("accepts supported MediaRecorder MIME parameters", () => {
27 |   assert.equal(allowedMime("audio/webm;codecs=opus"), "audio/webm");
28 | });
29 |
30 | test("rejects active document types", () => {
31 |   assert.throws(() => allowedMime("text/html"));
32 |   assert.throws(() => allowedMime("image/svg+xml"));
33 |   assert.throws(() => allowedMime("application/javascript"));
34 | });
```

=== package.json === (19 lines, 796 bytes)

```
1 | {
2 |   "name": "classconnect",
3 |   "private": true,
4 |   "type": "module",
5 |   "scripts": {
6 |     "test": "node --test backend/test/*.test.js",
7 |     "icons": "python3 scripts/make-icons.py",
8 |     "teacher": "node scripts/provision-teacher.mjs",
9 |     "db:local": "wrangler d1 migrations apply classconnect-db --local --config backend/wrangler.toml",
10 |    "db:remote": "wrangler d1 migrations apply classconnect-db --remote --config backend/wrangler.toml",
11 |    "dev:api": "wrangler dev --config backend/wrangler.toml",
12 |    "dev:web": "wrangler pages dev frontend --port 8788 --service API=classconnect-api",
13 |    "deploy:api": "wrangler deploy --config backend/wrangler.toml",
14 |    "deploy:web": "wrangler pages deploy frontend --project-name classconnect"
15 |  },
16 |   "devDependencies": {
17 |     "wrangler": "^4.0.0"
18 |   }
19 | }
```

=== .github/workflows/ci.yml === (25 lines, 343 bytes)

```
1 | name: CI
2 |
3 | on:
4 |   push:
5 |     branches: [main]
6 |   pull_request:
7 |     branches: [main]
8 |
9 | permissions:
10 |   contents: read
11 |
12 | jobs:
13 |   test:
14 |     runs-on: ubuntu-latest
15 |
16 |     steps:
17 |       - uses: actions/checkout@v4
18 |
19 |       - uses: actions/setup-node@v4
20 |         with:
21 |           node-version: "22"
22 |           cache: npm
23 |
24 |       - run: npm ci
25 |       - run: npm test
```

=== .github/workflows/deploy.yml === (39 lines, 767 bytes)

```
1 | name: Deploy ClassConnect
2 |
3 | on:
4 |   workflow_dispatch:
5 |
6 | permissions:
7 |   contents: read
8 |
9 | concurrency:
10 |   group: classconnect-production
11 |   cancel-in-progress: false
12 |
13 | jobs:
14 |   deploy:
15 |     runs-on: ubuntu-latest
16 |     environment: production
17 |
18 |     env:
19 |       CLOUDFLARE_API_TOKEN: ${ secrets.CLOUDFLARE_API_TOKEN }
20 |       CLOUDFLARE_ACCOUNT_ID: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
21 |
22 |     steps:
23 |       - uses: actions/checkout@v4
24 |
25 |       - uses: actions/setup-node@v4
26 |         with:
27 |           node-version: "22"
28 |           cache: npm
29 |
30 |       - uses: actions/setup-python@v5
31 |         with:
32 |           python-version: "3.x"
33 |
34 |       - run: npm ci
35 |       - run: npm test
36 |       - run: npm run icons
37 |       - run: npm run db:remote
38 |       - run: npm run deploy:api
39 |       - run: npm run deploy:web
```